

OpenSSL - kombinovaná forma studia

Knihovna OpenSSL realizuje protokol SSL, Secure Socket Layer (resp. TLS protokol), včetně příslušných kryptografických algoritmů vyžadovaných tímto protokolem. Aplikace openssl je utilita, jejíž prostřednictvím můžeme používat různé kryptografické funkce této knihovny, například:

- výpočet haše (otisku) zprávy
- generování klíče/klíčů
- šifrování/dešifrování, podepisování/ověřování podpisu
- práce s prvočísly
- vytvoření parametrů pro různé algoritmy (RSA, Diffie-Hellman a DSA, ...)
- tvorba X.509 certifikátů, práce se seznamem revokovaných certifikátů
- testy SSL/TLS klienta a serveru

Odkazy:

- <https://www.madboa.com/geek/openssl/#intro-version>
- <https://docs.openssl.org/master/man1/>

Odpovědi budete vkládat do odpovědního .pdf formuláře (viz LMS), který spolu s ostatními požadovanými soubory odevzdáte do 22.3. do 23:59 v LMS jako jeden .zip nebo .7z soubor (ten pojmenujte loginem řešitele).

1 Symetrická kryptografie

1. Vytvořte textový soubor (ASCII, bez diakritiky) o velikosti **nejméně 64 bajtů**. Můžete použít např. generátor Lorem ipsum
2. K šifrování/dešifrování vámi vytvořeného textového souboru použijte příkaz `openssl enc`.
3. Vyzkoušejte jeden symetrický algoritmus a 2 různé režimy (dle seznamu podporovaných algoritmů a režimů pro verzi openssl, se kterou pracujete).

Otázky: Který algoritmus a jaké režimy jste použili? Jaké příkazy a parametry jste použili?

2 Symetrická kryptografie - režimy

Pro pochopení vlastností různých šifrovacích režimů, proveďte:

1. Použijte soubor z předchozího příkladu.
2. Zašifrujte tento soubor pomocí AES-128 v režimech ECB, CBC (tj pracujeme s klíčem (i blokem) délky 128 bitů). Vzniknou dva šifrové soubory.

3. Pro oba šifrové soubory proveďte:
 - (a) Ovšem jeden bajt z druhého bloku (tj. např. někde od 3. bloku) šifrového textu byl porušen (byla změněna jeho hodnota - tj. změňte ji!). Pro tuto změnu použijte nějaký hex editor (GHex apod) nebo nějaký online hex editor (např. <https://hexed.it/>).
 - (b) Dešifrujte tento porušený šifrový text (za použití správného klíče, ve správném režimu (a použitím příslušného inicializačního vektoru je-li IV potřeba).
4. Tyto soubory odvezdejte - vstupní soubor s otevřeným textem, dále oba příslušné soubory s šifrovým textem, dva soubory s modifikovaným šifrovým textem a, pokud to půjde, tak dva dešifrované soubory (soubory s otevřeným textem po dešifrování).
5. **Pro oba šifrové soubory:** Zkuste místo změny hodnoty jednoho bajtu jeden bajt z šifrového textu úplně odstranit. Dešifrujte šifrový text. Co se stalo?

Otázky:

- (a) Kolik informace (tj. kolik otevřeného textu) bylo po dešifrování změněného šifrového textu (textu se změněným bajtem) porušeno, pokud byly použity režimy ECB, CBC? Stejně či rozdílné množství? Vysvětlete prosím, proč. Nejprve se pokuste odpovědět před provedením úlohy (tj. před dešifrováním), pak zkontrolujte správnost své odpovědi po provedení úkolu.
- (b) Co se stalo když jste místo změny hodnoty jednoho bajtu jeden bajt z šifrového textu úplně odstranili a pokusili se porušený šifrový text dešifrovat?

3 Hašovací funkce - generování haše

1. K hašování souboru libovolného typu a velikosti použijte příkaz `openssl dgst`.
2. Vyzkoušejte alespoň 3 různé algoritmy, porovnejte obdržené haše.

Otázky: Jaké příkazy a parametry jste použili? Jsou haše stejné? Jsou haše stejně dlouhé? A jak jsou jednotlivé haše dlouhé?

4 Hašovací funkce - náhodnost jednocestné hašovací funkce

1. Použijte soubor F_1 z předchozího příkladu.
2. Vygenerujte haš H_1 tohoto souboru F_1 pomocí funkce SHA256.
3. Změňte hodnotu jednoho bajtu vstupního souboru F_1 a uložte do souboru F_2 .
4. Vygenerujte haš H_2 modifikovaného souboru F_2 opět pomocí funkce SHA256.

Otázky:

- (a) Jsou haše H_1 a H_2 stejné nebo ne? (b) Kolik stejných bitů (bitů na dané pozici) mají haše H_1 a H_2 ? (c) Popište, jakým způsobem jste to určili (jaký příkaz jste použili nebo jakou aplikaci jste použili (vložte zdrojový kód), nebo např. jaké webové stránky). Nezajímá mne počet znaků, ale bitů!

5 Asymetrická kryptografie - generování klíčů

1. Vygenerujte si dvojici klíčů `openssl genrsa -out private.pem 1024`

Otázka (a): Jaká metoda, resp. jaký algoritmus byl použit pro generování klíčů?

2. Vypište si privátní klíč `cat private.pem`

Otázka (b): Jakou hlavičku a patičku má tento soubor, co z ní vyčteme?

3. Zobrazíme si oba klíče `openssl rsa -in private.pem -text`

Otázka (c): Jakou hodnotu má veřejný klíč?

4. Zašifrujte si soukromý klíč: `openssl rsa -in private.pem -des3 -out key3des.pem`

5. Podívejte (vypište si) se na veřejný klíč získaný takto: `openssl rsa -in private.pem -out public.pem -outform PEM -pubout`

Otázka (d): Co vyčteme z hlavičky a patičky?

6 Asymetrická kryptografie - digitální podpis

Použijte textový soubor z předchozích příkladů. Dále:

1. Vygenerujte dvojici RSA klíčů (veřejný a soukromý). Vyexportujte veřejný RSA klíč do souboru `public.pem`, který si zobrazte.
2. Vytvořte haš textového souboru pomocí SHA256. Podepište tuto haš (musíte sami zjistit jak na to); výsledek uložte do souboru (jméno např. `F1.sha256`).
3. Ověřte podpis v souboru `F1.sha256`.
4. Mírně změňte soubor `F1.sha256` (třeba záměnou hodnoty jednoho bajtu), a pak podpis ověřte.

Výše uvedené činnosti je potřeba provést pomocí vhodných příkazů a parametrů. Vysvětlete výsledky provedených operací.

Otázka: (a) Jaké příkazy a parametry jste použili? (b) Podařilo se Vám ověřit „originální“ podpis? (c) Podařilo se Vám ověřit podpis po změně podepsané haše `F1.sha256` a pokud, tak ne proč? Vysvětlete.

7 Asymetrická kryptografie - certifikáty

1. Vytvoření certifikační autority CA reprezentované certifikátem, který je podepsán soukromým klíčem této certifikační autority (selfsigned certificate):
vytvoření soukromého klíče (chráněného přístupovou frází):
`openssl genrsa -des3 -out CA.0.key 2048`
vytvoření žádosti o vydání certifikátu a jeho podepsání:
`openssl req -new -key CA.0.key -out CA.0.csr`
vytvoření certifikátu a jeho podepsání:
`openssl x509 -req -days 365 -signkey CA.0.key -in CA.0.csr -out CA.0.crt`
ověření certifikátu:
`openssl verify CA.0.crt`
převod certifikátu do čitelné podoby (zobrazte si ho):

```
openssl x509 -in CA.0.crt -text
```

Získejme z certifikátu např. údaje o jeho platnosti:

```
openssl x509 -noout -in CA.0.crt -dates
```

2. Vytvoření žádosti o certifikát pro subjekt PB a jeho vytvoření certifikační autoritou CA z předchozího bodu

- vytvoření soukromého klíče,

- vytvoření žádosti o vystavení,

- vytvoření a podepsání certifikátu soukromým klíčem CA z předchozího příkladu:

```
openssl x509 -req -days 365 -CA CA.0.crt -CAkey CA.0.key -in PB.0.csr  
-out PB.0.crt -CAcreateserial,
```

- ověření certifikátu (je třeba určit CA, která certifikát vydala).

Otázka: Jaký příkaz a parametry jste použili pro ověření certifikátu subjektu PB?

- převod certifikátu do čitelné podoby (zobrazte si ho).

3. Certifikáty mohou být uloženy v různých formátech PEM, DER ... (přečtěte si něco o těchto formátech). Chcete-li konvertovat z jednoho formátu na druhý, můžete použít `openssl` s argumenty `-inform a -outform`. Zkuste ...

Oba certifikáty, pro subjekt CA i PB, uložte do textových souborů a odevzdejte.